

2PROJECT INTERIM REPORT
ON
Electricity Bill Calculation System
(Using Core Java & OOPConcepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION2025-26

Submitted By
Poonam Rathia
RU-25-10973

Bachelor of Technology in Computer Science & Engineering
withAI and ML in association withGoogle

Under the Guidance of
Mr. Saumik Karmakar

SCHOOLOF COMPUTERSCIENCEAND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY
BHILAI, CG

(April 2026)

Acknowledgement:-

I would like to express my sincere gratitude to my teacher, **Mr. Saumik Karmakar**, for his valuable guidance, support, and encouragement throughout the development of my project titled **“Electricity Bill System.”**

I am also thankful to **Rungta International Skills University**, Bhilai for providing me with the opportunity and resources to work on this project. I extend my gratitude to the **AI and ML Association with Google** for their support and inspiration in enhancing my technical knowledge.

I would also like to thank my classmates of **SOCSE B.Tech CSE H Section** for their cooperation and support during the completion of this project.

Lastly, I express my heartfelt thanks to my parents and friends for their continuous motivation and encouragement.

Name: Poonam Rathia

Class: SOCSE B.Tech CSE) HSection

University: Rungta International Skills University

Project Title: Electricity Bill System

Teacher: Mr. Saumik Karmakar

Date: [17/06/2026]

INDEX

S. No.	Contents	Page No.
1.	Abstract	4
2.	Introduction	56
3.	Scope	7
4.	System Requirements	8
5.	Methodology	9
6.	Flowchart	10
7.	Class Diagram	11
8.	Implementation	12
9.	Sample Input / Output	13-14
10.	Future Enhancements	15
11.	Gantt Chart	15-16
12.	Conclusion	17
13.	References	18

Abstract

The Electricity Bill Calculation System is a Java-based project developed using Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and object interaction. The main aim of this project is to automate the process of calculating electricity bills based on electricity consumption using a slab-based tariff system. This project is needed to reduce manual calculations and minimize errors in bill generation. Traditional manual billing methods can be time-consuming and prone to mistakes, whereas a computerized system provides faster, more accurate, and reliable results. The use of OOP concepts improves code organization, reusability, and maintainability. The system consists of a Consumer class that stores consumer details such as ID, name, and units consumed, and a TariffCalculator class that contains the billing logic. Different tariff rates are applied according to predefined consumption slabs, and fixed charges are added to calculate the final bill amount. Encapsulation is implemented by keeping data members private and accessing them through public methods. The system generates accurate electricity bills and displays consumer information in a structured format. Overall, this project demonstrates the practical application of Java programming and OOP principles while providing an efficient solution for electricity bill management.

Introduction

In today's technology-driven world, automation plays an important role in reducing manual effort and improving accuracy. The Electricity Bill Calculation System is developed using Java and is based on Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and object interaction. The purpose of this project is to automate the process of calculating electricity bills using a slab-based tariff system.

The system is designed around a Consumer class that stores important consumer information such as consumer ID, name, and units consumed. Another class, TariffCalculator, contains the business logic required to calculate electricity charges according to different tariff slabs. This separation of data and logic improves code organization and maintainability.

One of the key features of this system is its multi-tier billing mechanism, where different rates are applied to different ranges of electricity consumption. The program automatically calculates the charges for each slab, adds fixed charges, and generates the final bill amount.

Encapsulation is implemented by keeping data members private and providing controlled access through public getter methods.

The system provides accurate and error-free bill calculation while demonstrating the practical application of OOP concepts. Overall, this project serves as a simple, efficient, and scalable solution for electricity bill management and

helps in understanding real-world Java programming techniques.

Features of the Project

- Automated electricity bill calculation
- Multi-tier slab-based tariff system
- Consumer data management using classes and objects
- Data security through encapsulation
- Object interaction between Consumer and TariffCalculator classes
- Accurate and error-free bill generation
- Simple and user-friendly console-based interface

Scope

This project is suitable for small electricity billing applications and educational purposes where electricity charges need to be calculated accurately and efficiently. It provides a simple system to store consumer information and generate electricity bills automatically using a slab-based tariff structure.

The current system works in a console-based environment and demonstrates core Object-Oriented Programming concepts such as classes, objects, encapsulation, and object interaction. It is ideal for academic learning and small-scale implementations where a lightweight and easy-to-use solution is required.

In the future, this project can be enhanced by integrating a database to store consumer records permanently.

Additional features such as a Graphical User Interface (GUI), online bill payment, bill history tracking, receipt generation, and customer account management can also be added to make the system more advanced, efficient, and user-friendly.

System Requirements

Hardware Requirements:

- Computer or Laptop
- Minimum 4 GB RAM
- Intel/ AMD Processor

Software Requirements:

- Java JDK 8 or higher
- VS Code, Eclipse, or any Java IDE
- Windows/ Linux Operating System

Methodology

The program works in a simple step-by-step process.

1. **Create Consumer Object**

The program starts by creating a Consumer object containing the consumer's ID, name, and units consumed.

2. **Create TariffCalculator Object**

A TariffCalculator object is created to handle the electricity bill calculation process.

3. **Pass Consumer Object**

The Consumer object is passed to the generateBill() method of the TariffCalculator class.

4. **Apply Tariff Slab Rates**

The calculator checks the number of units consumed and applies the appropriate tariff rates:

- o 0–100 units → ₹1.5 per unit
- o 101–200 units → ₹2.5 per unit
- o 201–300 units → ₹4.0 per unit
- o Above 300 units → ₹6.0 per unit

5. **Calculate Bill Amount**

The charges for each slab are calculated and added together to obtain the total energy charge.

6. **Add Fixed Charge**

A fixed charge of ₹125 is added to the calculated energy charge.

7. **Display Final Bill**

The program displays the consumer details along with the final electricity bill amount .

Flowchart

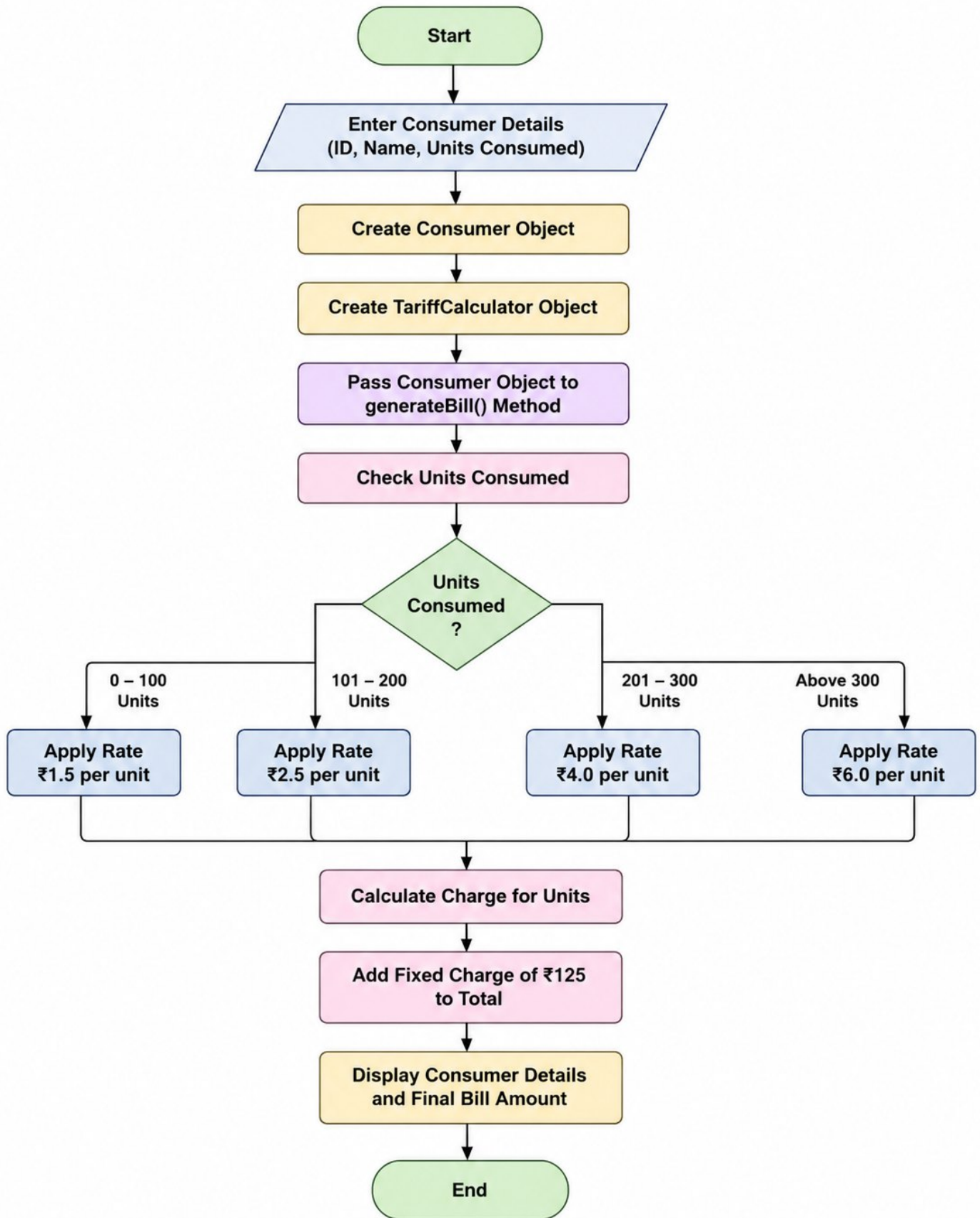


Figure 1: Flowchart of Electricity Bill Calculation System

Class Diagram

Class Diagram – Electricity Bill Calculation System

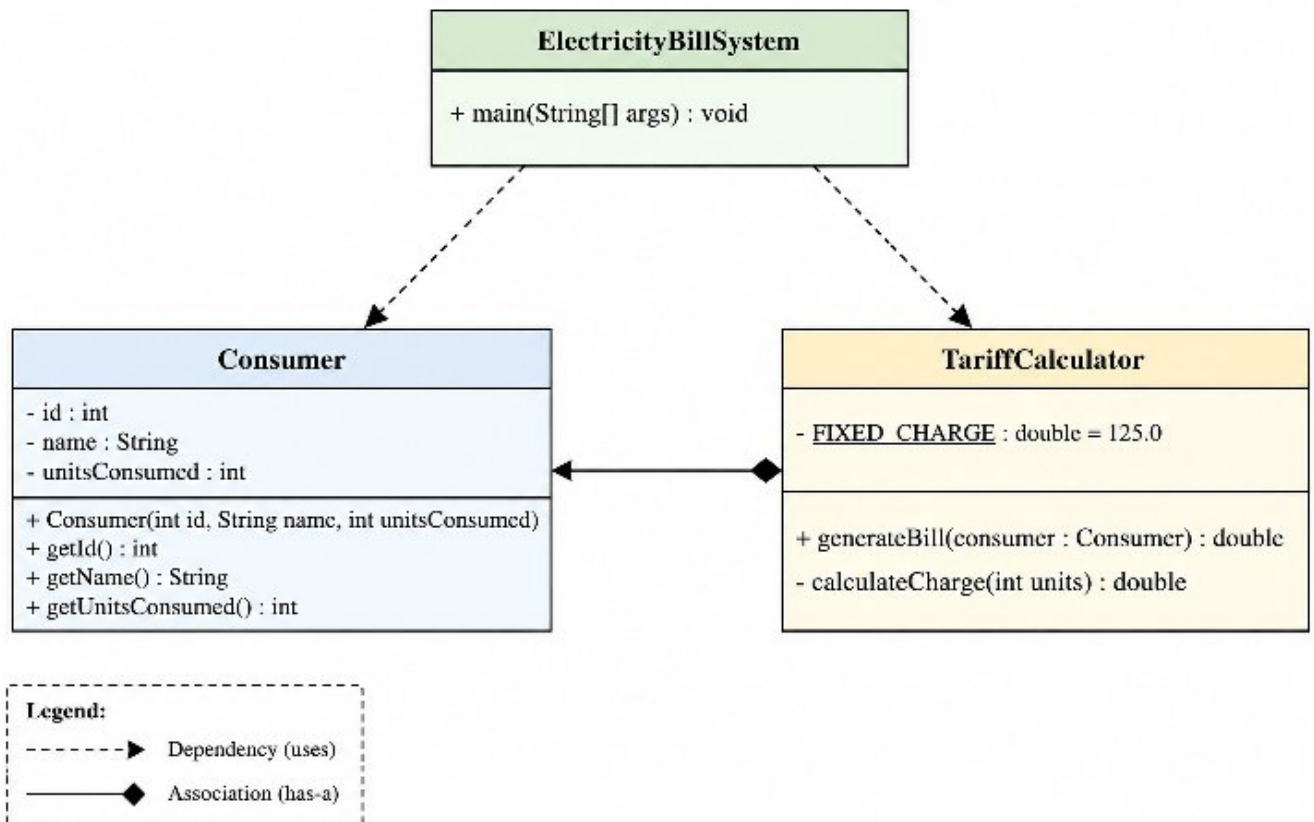


Figure 2 : Class Diagram of Electricity Bill Calculation System

Implementation

Consumer Class

The Consumer class is used to store consumer information such as consumer ID, name, and units consumed. These attributes are declared as private to ensure data security through encapsulation. The class provides a constructor to initialize the values and getter methods to access the data safely. It acts as a data container and does not perform any billing calculations.

TariffCalculator Class

The TariffCalculator class contains the complete electricity billing logic. It includes the generateBill() method, which accepts a Consumer object and calculates the bill based on predefined tariff slabs. The method applies different rates for different ranges of electricity consumption and adds a fixed charge to generate the final bill amount. This class separates business logic from data management, making the program more organized and maintainable.

displayBill() Method

The displayBill() method is responsible for presenting the bill details to the user. It calls the generateBill() method to calculate the bill and then displays the consumer ID, consumer name, units consumed, and total electricity bill in a structured format. This method improves the readability of the output.

ElectricityBillSystem Class (Main Class)

The ElectricityBillSystem class is the main class of the program. It contains the main() method, which acts as the entry point of the application. This class creates objects of Consumer and TariffCalculator, passes the Consumer object to the calculator, and displays the final bill. It demonstrates object creation,¹³ object interaction, and the

practical implementation of Object-Oriented Programming concepts in Java.

Sample Input/Output

Sample Input-

Consumer ID: 101
Consumer Name: Rahul
Units Consumed: 250

Sample Output-

Consumer ID: 101
Name: Rahul
Units Consumed: 250
Total Electricity Bill: ₹725.00

```

1 // This class is a blueprint for consumer data
2 class Consumer {
3     // Private data for security
4     private int id;
5     private String name;
6     private int unitsConsumed;
7
8     // Constructor to create object
9     public Consumer(int id, String name, int unitsConsumed) {
10         this.id = id;
11         this.name = name;
12         this.unitsConsumed = unitsConsumed;
13     }
14
15     // Public methods to get data safely
16     public int getId() { return id; }
17     public String getName() { return name; }
18     public int getUnitsConsumed() { return unitsConsumed; }
19 }
20
21 // This class has the billing logic
22 class TariffCalculator {
23
24     // This method takes a Consumer object and returns the bill
25     public double generateBill(Consumer consumer) {
26         int units = consumer.getUnitsConsumed();
27
28         // Check for wrong input
29         if (units < 0) {
30             System.out.println("Error: Units cannot be negative");
31             return 0;
32         }
33
34         // Calculate bill
35         double bill = units * 3;
36
37         // Return bill
38         return bill;
39     }
40 }
41
42 // Main class
43 public class ElectricityBillSystem {
44     public static void main(String[] args) {
45         // Create consumer object
46         Consumer consumer = new Consumer(101, "Rahul", 250);
47
48         // Create tariff calculator object
49         TariffCalculator calculator = new TariffCalculator();
50
51         // Generate bill
52         double bill = calculator.generateBill(consumer);
53
54         // Print bill
55         System.out.println("Total Electricity Bill: " + bill);
56     }
57 }

```

ElectricityBillSystem
Consumer ID: 101
Name: Rahul
Units Consumed: 250
Total Electricity Bill: 725.0

[Done] exited with code=0 in 1.545 seconds

```

2 class Consumer {
3     // Constructor to create object
4     public Consumer(int id, String name, int unitsConsumed) {
5         this.id = id;
6         this.name = name;
7         this.unitsConsumed = unitsConsumed;
8     }
9
10    // Public methods to get data safely
11    public int getId() { return id; }
12    public String getName() { return name; }
13    public int getUnitsConsumed() { return unitsConsumed; }
14 }
15
16 // This class has the billing logic
17 class TariffCalculator {
18
19     // This method takes a Consumer object and returns the bill
20     public double generateBill(Consumer consumer) {
21         int units = consumer.getUnitsConsumed();
22
23         // Check for wrong input
24         if (units < 0) {
25             System.out.println("Error: Units cannot be negative");
26             return 0;
27         }
28
29         // Calculate bill
30         double bill = units * 3;
31
32         // Return bill
33         return bill;
34     }
35 }
36
37 // Main class
38 public class ElectricityBillSystem {
39     public static void main(String[] args) {
40         // Create consumer object
41         Consumer consumer = new Consumer(101, "Rahul", 250);
42
43         // Create tariff calculator object
44         TariffCalculator calculator = new TariffCalculator();
45
46         // Generate bill
47         double bill = calculator.generateBill(consumer);
48
49         // Print bill
50         System.out.println("Total Electricity Bill: " + bill);
51     }
52 }

```

Warning: Powershell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run "Import-Module PSReadLine".

PS C:\Users\Nishant\Desktop> java ElectricityBillSystem.java
Consumer ID: 101
Name: Rahul
Units Consumed: 250
Total Electricity Bill: 725.0
PS C:\Users\Nishant\Desktop>

Figure 3: Output of Electricity Bill Calculation System

Future Enhancements

- Add a database to store consumer records permanently.
- Develop a graphical user interface (GUI) using Java Swing or JavaFX.
- Generate electricity bills in PDF format.
- Add tax calculation and discount facilities.
- Maintain bill payment history for consumers.
- Support multiple consumers in a single system.
- Add user login and authentication features.
- Enable online bill payment integration.

Gantt Chart

Task	Week 1	Week 2	Week 3	Week 4
Planning & Structure				
Flowchart & Class Diagram				
Coding & Implementation				
Testing & Debugging				
Documentation & Report Writing				
Final Review & Submission		16		

1. Planning & Structure

The project began with planning and designing the overall structure of the Electricity Bill Calculation System. The required classes, namely Consumer and TariffCalculator, were identified and their responsibilities were clearly defined. The tariff slabs, billing logic, flowchart, class diagram, and program workflow were planned before starting the coding phase. This helped in developing a well-organized and structured application.

1. Time Management

The Gantt chart helped in organizing project activities according to a fixed timeline. Different tasks such as planning, flowchart and class diagram preparation, coding, testing, documentation, and final submission were completed within their assigned weeks. This ensured smooth progress of the project, improved productivity, and helped avoid delays during development. Proper time management also made it easier to complete the project systematically and efficiently .

Conclusion

The Electricity Bill Calculation System successfully demonstrates the application of Java and Object-Oriented Programming (OOP) concepts in solving a real-world problem. The project uses encapsulation to protect consumer data and object interaction to perform electricity bill calculations efficiently. The slab-based billing system generates accurate results and works correctly for different levels of electricity consumption.

This project helped in understanding the importance of separating data management from business logic, resulting in a more organized and maintainable program. Although the system is simple and console-based, it is fully functional and effectively demonstrates key OOP concepts such as classes, objects, encapsulation, and modular design. The project can be further enhanced with features such as database integration, graphical user interfaces, and online bill payment facilities in the future.

References

Book

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

Website

[2] Oracle, "Java Documentation," Oracle. [Online]. Available: <http://docs.oracle.com/javase/tutorial/>

Conference Paper

[3] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River: Prentice Hall, 2006.

Journal Article

[4] GeeksforGeeks, "Encapsulation in Java," GeeksforGeeks [Online]. Available: <https://www.geeksforgeeks.org/java/encapsulation-in-java/>